TP N°1

Suites numériques

Dans tout le TP, on utilisera les bibliothèques suivantes. Les questions marquées PAPIER se traitent sur feuille, celles marquées PYTHON sur machine.

TP1.py

```python
import math, random, sys
import numpy as np
import matplotlib.pyplot as plt
import sympy as sp
sys.set_int_max_str_digits(0)
```

1 PROBLÈME — Autour de la formule de Stirling

La factorielle d'un entier naturel n intervient partout : en probabilités et en dénombrement, en informatique, en physique statistique, etc.
Le problème est que $n!$ devient très vite gigantesque : 10! vaut déjà plus de trois millions, et 100! possède 158 chiffres. Il est donc naturel de chercher une **formule simple** qui donne l'ordre de grandeur de $n!$ lorsque n est grand. C'est l'objet de la **formule de Stirling** :

$$n! \underset{n\to+\infty}{\sim} \sqrt{2\pi n}\left(\frac{n}{e}\right)^n.$$

Dans ce problème, nous allons **redécouvrir cette formule expérimentalement** à l'aide de Python :

1. nous commencerons par programmer la factorielle et observer sa croissance ;
2. nous étudierons ensuite le logarithme de $n!$ et conjecturerons un premier équivalent ;
3. enfin, en analysant l'erreur commise, nous ferons apparaître le facteur $\sqrt{n}$, puis la constante $\sqrt{2\pi}$.

Tout au long du problème, on gardera en tête qu'une observation numérique, aussi convaincante soit-elle, **n'est pas une démonstration** : Python permet de conjecturer, les mathématiques permettent de

PARTIE 1 — Divergence de $n!$

PAPIER **Question 1** – Rappeler la définition de $n!$ pour n un entier naturel.

PYTHON **Question 2** – Écrire une fonction `fact(n)` qui calcule $n!$ avec une boucle `for`. La comparer avec `math.factorial` pour $n = 0, 1, \ldots, 20$.

PYTHON **Question 3** – Combien 1000! a-t-il de chiffres ? Et 10 000! ?

> **Indication** On pourra utiliser les fonctions `len` et `str`.

PYTHON **Question 4** – À l'aide d'un script Python, illustrer le fait que $n! \underset{n\to+\infty}{\longrightarrow} +\infty$.

PARTIE 2 — Un premier équivalent : $\ln(n!) \sim n\ln(n)$

PAPIER **Question 5** – Soit $k \in \mathbb{N}^*$ et $t \in [k; k+1]$. Justifier que $\ln(k) \leqslant \ln(t) \leqslant \ln(k+1)$.

PAPIER **Question 6** – En déduire que pour $k \geqslant 2$,

$$\int_{k-1}^{k} \ln(t)\,\mathrm{d}t \leqslant \ln(k) \leqslant \int_{k}^{k+1} \ln(t)\,\mathrm{d}t.$$

PAPIER **Question 7** – En sommant l'inégalité précédente pour k allant de 2 à n, montrer que pour tout $n \geqslant 1$,

$$n\ln(n) - n + 1 \leqslant \ln(n!) \leqslant (n+1)\ln(n+1) - n.$$

Indication On rappelle qu'une primitive de ln sur $\mathbb{R}_+^*$ est $x \mapsto x\ln(x) - x$.

PAPIER **Question 8** – Montrer que $n\ln(n) - n + 1 \sim n\ln(n)$ et $(n+1)\ln(n+1) - n \sim n\ln(n)$ puis conclure.

Indication On pourra s'inspirer de la fin de l'exercice 6 du TD3.

PYTHON **Question 9** – À l'aide d'une liste en compréhension, construire la liste `X` des entiers 10^{2^k} pour $k = 0, 1, \ldots, 8$, c'est-à-dire

$$X = \left[10,\ 10^2,\ 10^4,\ 10^8,\ \ldots,\ 10^{256}\right].$$

PYTHON **Question 10** – On admet que `math.lgamma(n+1)` renvoie la valeur de $\ln(n!)$, sans calculer $n!$. À l'aide de `math.lgamma` et `math.log`, construire la liste `Y` des nombres $\frac{\ln(n!)}{n\ln(n)}$ pour n dans la liste `X` de la question 9.

PYTHON **Question 11** – À l'aide de la fonction `plt.plot`, représenter graphiquement les termes de la liste `Y` à l'aide de points, **sans utiliser la liste** `X` en abscisse. Tracer également en rouge la droite d'équation $y = 1$ à l'aide de la fonction `plt.axhline`. Que peut-on constater ? Est-ce en accord avec les questions précédentes ?

PARTIE 3

Mesure de l'erreur

On a vu que $\ln(n!) \sim n\ln(n) - n$. On s'intéresse à la suite (e_n), l'erreur, définie par

$$e_n = \ln(n!) - (n\ln(n) - n).$$

PYTHON **Question 12** – Tracer e_n en fonction de n pour $n = 1, 2, \ldots, 10^5$. On fera encore appel aux fonctions `math.lgamma` et `math.log`.

PYTHON **Question 13** – Tracer e_n en fonction de $\ln(n)$ pour $n = 1, 2, \ldots, 10^5$. On fera appel à la fonction `math.log`. Quelle semble être la nature de la courbe obtenue ? En déduire une conjecture de la forme $e_n \approx \alpha\ln(n) + \beta$, où α et β sont des réels à estimer graphiquement.

PAPIER **Question 14** – À l'aide de la conjecture de la question précédente, en déduire une approximation de $n!$ pour n grand. On l'écrira sous la forme $n! \approx C\sqrt{n}\left(\frac{n}{e}\right)^n$, où C est une constante à déterminer numériquement. Comparer C^2 à 2π.

2 PROBLÈME
Suites récurrentes et chaos

De nombreux phénomènes évoluent par étapes : chaque état se déduit du précédent par une règle fixe. On les modélise par des **suites récurrentes** de la forme $u_{n+1} = f(u_n)$. C'est le cas, par exemple, de l'évolution d'une population d'une année sur l'autre, de la propagation d'une épidémie ou de certains algorithmes itératifs.

Dans ce problème, nous étudions l'une des plus célèbres : la **suite logistique**, introduite en biologie pour modéliser une population animale. Pour $r \in [0,4]$, on pose $f_r(x) = r\,x(1-x)$ et

$$u_0 \in [0,1], \qquad u_{n+1} = r\,u_n(1-u_n).$$

Ici, u_n représente la population l'année n (normalisée entre 0 et 1) et r le taux de reproduction. Le terme $1-u_n$ traduit le fait que, lorsque la population devient trop importante, les ressources manquent et la croissance ralentit.
Malgré sa formule très simple, cette suite a un comportement étonnamment riche selon la valeur de r :

1. nous commencerons par observer numériquement la suite pour différentes valeurs de r ;
2. nous démontrerons ensuite sa convergence dans deux cas particuliers ;
3. nous visualiserons son comportement à l'aide des diagrammes en toile d'araignée ;
4. enfin, nous découvrirons le **chaos** : une suite parfaitement déterministe peut devenir imprévisible, et une infime modification de u_0 peut changer complètement son évolution.

PAPIER **Question 15** – Étudier les variations de f_r sur $[0,1]$ et montrer que pour tout $x \in [0;1]$, $f_r(x) \in [0;1]$. En déduire que pour tout $n \in \mathbb{N}$, $u_n \in [0,1]$.

PYTHON **Question 16** – Écrire une fonction `suite(r, u0, N)` qui prend en argument le réel r, le premier terme u_0 et un entier N, et qui renvoie la liste $[u_0, u_1, \ldots, u_N]$ des $N+1$ premiers termes de la suite.

PYTHON **Question 17** – On prend $u_0 = 0{,}2$. À l'aide de la fonction `plt.plot`, représenter les 50 premiers termes de la suite pour chacune des valeurs $r \in \{0{,}5\,;\ 2\,;\ 3{,}2\,;\ 3{,}5\,;\ 3{,}9\}$. On utilisera `plt.ylim(0, 1)` pour fixer la même échelle verticale pour les cinq graphiques.
Décrire le comportement de la suite dans chaque cas.

PAPIER **Question 18** – On suppose dans cette question que $r \in [0,1[$.

1. À l'aide de la question 15, montrer que pour tout $n \in \mathbb{N}$, $0 \leqslant u_{n+1} \leqslant r\,u_n$.
2. À l'aide d'une récurrence, montrer que pour tout $n \in \mathbb{N}$, $0 \leqslant u_n \leqslant r^n u_0$.
3. En déduire la limite de la suite (u_n).

PAPIER **Question 19** – On suppose dans cette question que $r = 2$ et $u_0 \in]0,1[$. On pose, pour tout $n \in \mathbb{N}$, $w_n = 1 - 2u_n$.

1. Montrer que pour tout $n \in \mathbb{N}$, $w_{n+1} = w_n^2$.
2. À l'aide d'une récurrence, montrer que pour tout $n \in \mathbb{N}$, $w_n = w_0^{2^n}$.
3. En déduire la limite de la suite (w_n) puis celle de (u_n).

Pour visualiser le comportement d'une suite récurrente $u_{n+1} = f_r(u_n)$, on utilise un **diagramme en toile d'araignée**. On trace sur un même graphique la courbe $y = f_r(x)$ et la droite $y = x$, puis on construit les termes de la suite graphiquement :

— partant du point (u_0, u_0) situé sur la droite $y = x$, on trace un segment **vertical** jusqu'à la courbe : on arrive au point (u_0, u_1) ;
— on trace ensuite un segment **horizontal** jusqu'à la droite $y = x$: on arrive au point (u_1, u_1) ;
— on recommence à partir de (u_1, u_1), et ainsi de suite.

PYTHON **Question 20** – Écrire une fonction `toile(r, u0, N)` qui trace sur un même graphique la courbe de f_r sur $[0,1]$, la droite $y = x$, ainsi que le diagramme en toile d'araignée des N premiers termes de la suite. On utilisera les fonctions `np.linspace` et `plt.plot`.

PYTHON **Question 21** – Tracer le diagramme en toile d'araignée avec $u_0 = 0{,}2$ et $N = 60$ pour chacune des valeurs $r \in \{2{,}8\,;\ 3{,}3\,;\ 3{,}9\}$. Interpréter chacun des diagrammes obtenus.

PYTHON **Question 22** – On se place dans le cas $r = 3{,}3$. On note a et b les deux derniers termes de la liste `suite(3.3, 0.2, 60)`. Calculer $f_r(a)$ et $f_r(b)$. Que constate-t-on ?

PYTHON **Question 23** – On se place dans le cas $r = 3{,}9$. On considère deux suites (u_n) et (v_n) vérifiant la même relation de récurrence, mais de premiers termes très proches : $u_0 = 0{,}2$ et $v_0 = 0{,}2 + 10^{-10}$.

1. Calculer les 60 premiers termes de chacune des deux suites.
2. Représenter $\ln(|u_n - v_n|)$ en fonction de n.
3. Que constate-t-on ?

Les questions précédentes montrent que le comportement de la suite dépend fortement de r : convergence, cycles, chaos... Pour avoir une vue d'ensemble, on trace le **diagramme de bifurcation**. Pour chaque valeur de r, on calcule un grand nombre de termes de la suite, on oublie les premiers (le temps que la suite « s'installe ») et on place les suivants sur un graphique, à l'abscisse r.

PYTHON **Question 24** – Pour 500 valeurs de r régulièrement espacées dans $[2{,}5\,;4]$, calculer les 600 premiers termes de la suite avec $u_0 = 0{,}2$, puis représenter les 100 derniers par des points d'abscisse r. On utilisera les fonctions `np.linspace` et `plt.plot`, avec le style `","` qui trace des points de la taille d'un pixel.

PAPIER **Question 25** – À l'aide du diagramme de bifurcation obtenu à la question 25, répondre aux questions suivantes.

1. Pour chacune des valeurs $r \in \{2{,}8\,;\ 3{,}3\,;\ 3{,}5\,;\ 3{,}9\}$, combien de points voit-on sur la verticale d'abscisse r ? Faire le lien avec les questions précédentes.
2. Pour quelles valeurs de r la suite semble-t-elle converger ? Sur quelle courbe se situent alors les